

B13705023 蔡承叡 HW11

1. 三角準則的侵略者!?

(a)

1. 2016 年 10 月，Dyn(一個很大的 DNS provider)，承受了一個每秒 1 Tb 的 DDoS 攻擊，造成許多網站像是 Twitter，Netflix 等下線。這攻擊違反了 CIA 中的 A (Availability)，因為系統被癱瘓，使得合法使用者無法使用服務。
2. 2017 年的 WannaCry ransomware 利用了 Windows SMB 漏洞造成了很大的傷害，其中違反了 CIA 中的 A，因為檔案被加密，系統 (人)無法使用。I (Integrity) 也有可能受到影響，因為檔案被加密其實就是沒有經過授權的變更，所以完整性應該還是有些影響。

參考資料：

<https://www.corero.com/famous-ddos-attacks/>

(b)

- Assumptions: 攻擊者可以接觸筆電，不知道密碼但有基礎知識，筆電會要求登入認證

Threat Model	Countermeasure
攻擊者實際取得筆電，想在登入畫面 brute force 破解密碼，或是用 USB boot 進去	設定重複嘗試次數上限，達到上限後傳送通知給擁有者，且關閉 USB boot 功能
攻擊者偷偷在使用者背後看他輸入的密碼 (shoulder surfing)	用生物辨識登入

(c)

- Assumptions: 場所提供的 QR code 正確對應到該場所的代碼，大家都有經實名登記的門號。

Threat Model	Countermeasure
A 幫朋友 B C 連續掃描，導致一人幫多人登記的情形，而 B C 沒有登記	系統限制單一門號在短時間內只能成功登記一次，或是如果偵測到多次登記則要求核對人數
A 跟朋友 B 借用 SIM 卡，拿來仿冒本人使用	有新的 sim 啟用之後或掃 qr code 之後本人需要完成生物辨識才可以使用/進入場所

(d)

- Assumptions: 學生用電腦考試，有固定的考試用帳號

Threat Model	Countermeasure
學生用第二台裝置或透過 dc/line 與非同組人員交換答案	強制開啟螢幕鏡頭，且全程分享螢幕
學生偷戴藍芽耳機，跟其他組通話	強制開麥錄音，以及每隔 20 分鐘突襲檢查

參考資料：

<https://ithelp.ithome.com.tw/articles/10370407>

2. 果汁店也有洞！

(a)

(i) DOM XSS

- 解題過程：發現右上角的 search field 的東西在送出後會出現在 html 之中，把 `<iframe src="javascript:alert( xss )">` 丟進搜尋欄送出，就可以看到成功把 js 注入的畫面 (跳 error)
- 漏洞類型原理：DOM型的XSS攻擊，是在JS載入網頁時，直接在網頁中嵌入夾帶惡意程式碼的DOM物件進行攻擊。
- 解決漏洞方法：在能夠讓使用者輸入資料的區域都要做 escape，就是如果有一些程式語言的特殊字元符號 `<` or `"`，都用 escape character 代替掉。

(iii) Login MC SafeSearch

- 解題過程：在 Juice shop permafrost edition 看到 MC Safesearch 的 email：
mc.safesearch@juice-sh.op，接著去聽他的歌 "Protect Ya Passwordz" 裡面可以聽到他說："Mine's my dog, Mr. Noodles. It don't matter if you know because I was tricky and replaced some vowels with zeroes." 直接猜密碼是 Mr. N00dles，還真的是
- 漏洞類型原理：Sensitive data exposure，原理如其名，就是把敏感的資料公開
- 解決漏洞方法：不要在社交媒體上公開自己的個資

(iv) Five-Star Feedback

- 解題過程：從 admin 帳號進入，去 <http://localhost:3000/#/administration> 把右邊的五星評論刪除即可
- 漏洞類型原理：Broken access control，若沒有管好如何登入 admin 帳號，那麼就可以隨意進入管理頁面刪除資料。
- 解決漏洞方法：避免任何人能夠登入 admin 帳號，選個高強度密碼，用 input sanitization 避免 SQLi。也要 apply least privilege，只給予人們最低權限需求。

(v) View Basket

- 解題過程：在兩個不同帳號隨便加東西進去 basket，然後在 burp 開啟 intercept 之後，點選 basket 的符號，攔截封包看到上面的 URL 有去 get 個 basket/2，把後面的 2 換成 1 重新 forward 就可以 access 別人的貨了！
- 漏洞類型原理：Broken access control，準確地說是 Insecure Direct Object Reference (IDOR)，系統把購物籃的識別代碼直接暴露在 url 之中。
- 解決漏洞方法：後端做權限認證，目前登入的 userID 是否和 basketID 相同。而且不應該在 url 直接暴露可以預測的 ID，可以用 session token 來代替之類的。

(vi) Password Strength

- 解題過程：去 reviews 發現 admin 的 email 叫做 admin@juice-sh.op，密碼亂試試到 admin123 是對的。或是我有發現另一種解法，就是在 cookies 竟然有 password 的 hash "password":"0192023a7bbd73250516f069df18b500"，把這個拿去網路上 crackstation 炸他，也可以得到 admin123 這個密碼。
- 漏洞類型原理：Broken authentication，密碼選得太差就會有這個問題。或是 hash algorithm 不夠強大，用 md5 挺危險的
- 解決漏洞方法：也是使用高強度密碼 (記不起來就用 password manager) 或是換成比較強的 hash function 像是 sha-256

(b)

- SSRF (Server Side Request Forgery): 攻擊者讓伺服器發起請求，打其他外網原本不能存取的地方。網頁後端在呼叫 API 時，反而在未經授權地呼叫、請求了其他的資源。
- CSRF (Cross-site Request Forgery): 攻擊者讓使用者在登入狀態之下做不想做的動作，具體來說是讓用戶 C 在 A 網站登入，然後去到一個惡意網站 B，而 B 收到請求之後利用 C 的 Cookie 回發送一個惡意請求訪問 A (A 不知道是 B 還是 C 發過來的)
- XSS (Cross-site scripting): 攻擊者把 JS (或其他 script) 注入網頁的輸出中，當其他使用者訪問頁面的時候，腳本會在用戶的瀏覽器執行，執行一些惡意操作像是偷取 cookie，對話訊息等等。

	SSRF	CSRF	XSS
誰發請求	伺服器	受害者的瀏覽器	瀏覽器執行攻擊者 js
登入認證？	不用	要	不用

參考資料：

<https://blog.csdn.net/VN520/article/details/137006452>

<https://ithelp.ithome.com.tw/articles/10324329>

<https://ithelp.ithome.com.tw/articles/10313080>

<https://medium.com/@codingboltacademy/idor-broken-access-control-efeb6698919f>

與蔡冠毅同學討論

3. R-SA ! 破密部

(a)

- 看了一下，因為 Anon 只吃 name=soyo 的簽章，但 soyo 又不簽 name=soyo，所以要想辦法給 soyo 簽一個 name=soyo 加密後的 string，然後再反向推回目標簽章。
- 先去拿 soyo 的 public key，把 e,n 存起，因為目標是我們想要加密後的 name=soyo signature，所以先 blind sign 這個 message，使用這個公式 $m = T \cdot r^e \pmod n$ ，T 是 bytes_to_long("name=soyo") (因為 RSA 都是在整數上運行的)，而 r 選的是讓 $\gcd(r, n) = 1$ (不然 unblinding 就做不了)。

- 接下來得到這個整數之後，看到 `soyo.py` 裡面 signature 是要吃 bytes 的，所以把 int 轉 bytes
- 然後用 socket 把這個 bytes 丟進去讓 soyo 簽名，得到一大串東西把他叫 s_m (一定要用 socket，手動輸入會說 message too long)
- 所以真正要給 Anon 看 (也就是真正 name=soyo 的簽名) 就是 $s = s_m \cdot r^{-1} \pmod n$ 。得到 s 之後就可以跟 Anon 聯繫，輸入 ID: name=soyo 跟這個 s 就可以拿到 flag 了！

```
b13705023@nasaws1:~/CTF$ nc 140.112.187.51 11451
Hello, I'm Anon (not Ann). Sorry... I only talk to Soyo now.
It seems that I don't know who you are, show me your ID in the format "name=<your_name>" and give me its signature signed by your RSA private key.
ID: name=soyo
Signature: 3803959996153342782051100522215368947846178800677769541130827176385637692859433759524
468683690309996602357802688288635766424960365714856579638045783810888998835051429139495846699422
859544629234432651118956658305018934611191080278484445503388645081522105730006403780867521855823
013977640403821885570826698890903593197819100659234303813965977961973558663566211247968141899137
776763775253792074268441226101190177652032504837154881393696166343148747427327615329311428867290
617582262390939737133978019301754141096575086635370216068122696550465119199835213496959438094950
210517042605159990059243217400679240234312534968831772569656089671449440677765695049030504412496
745550087259623604447022100631788579095776429574058721283145383269246110990012953825817398693687
077660939045157658309403464746934018353380035313551306040999234139766367149700590515451830257486
421516456785102464830922253260802508680065744676112875579605028210611969801844452271482474330381
376277456058320235098052102109584980366350460591177664049805754919213498270543809662578072969794
415017082146616293962175817285226806738224825878104016581106924352241285803228641336926368415887
73679430917974617479986940364472463827718582982803499051112387323430689834414800641154005658
I'm sure you are Soyorin since your signature matches the ID :D
Here is the flag just for you, Soyorin: NASA_HW11{blind_signing_is_dangerous}
Hold on, let me generate my RSA key.
```

- FLAG: NASA_HW11{blind_signing_is_dangerous}

參考資料：

https://en.wikipedia.org/wiki/Blind_signature

(b)

原本想試試能不能直接對 diary 開 e 次根號然後再直接 decode，但失敗了 (因為 $m^7 > n$ ，會跑到 mod)

- 上網查資料後發現對於小 public exponent，可以使用 Hastad's Broadcast Attack
- 我們已經有 soyo 用 private key 簽過的 name=soyo 了，所以可以無限讓 anon 生出不同的 public key 以及 encrypted diary
- 我重複生 7 組，現在就有 $c_i = m^7 \pmod{n_i}$ for i from 1 to 7
- 現在 CRT (Chinese Remainder Theorem) 就派得上用場了，因為它能让我把這 7 個 mod 完的碎片拼成完整的 $X = m^7$ 且 $m^7 < n_1 n_2 \dots n_7$

- 所以只要把這個 X 開 7 次根號就可以得到 m

```
PS C:\Users\ohwow\Desktop\b13705023> & C:/Users/ohwow/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/ohwow/Desktop/b13705023/code/hastad atk.py"
b"Anon's Secret Diary: Today, I went to a Mister Donut nearby and met Sumita Mana from
• Sumimi! She assured me it's safe to use a small public exponent in RSA encryption becau
se RSA is super secure. I trust her completely, as she wouldn't lie to me! By the way,
my secret flag is: NASA_HW11{W0w_y0u_kNow_h@st@d'5_bro4dc@s7_47t@cK}"
PS C:\Users\ohwow\Desktop\b13705023> █
```

- FLAG: NASA_HW11{W0w_y0u_kNow_h@st@d'5_bro4dc@s7_47t@cK}

參考資料：

<https://stackoverflow.com/questions/19255120/is-there-a-short-hand-for-nth-root-of-x-in-python>

<http://koclab.cs.ucsb.edu/teaching/cren/project/2017/chennagiri.pdf>

<https://xanhacks.gitlab.io/ctf-docs/crypto/rsa/08-hastad-broadcast-attack/>

<https://www.youtube.com/watch?v=ru7mWZJIRQg>

4. TESTING in the FUZZ

(a)

原理差異

1.
 - Mutation-based: 從有效的 seed 輸入，真實檔案，網路請求，API call 等等的下手，做的是透過位元翻轉，插入，刪除重排等變異生成測資
 - Generation-based: 是根據一個固定的格式 (protocol，語法，grammar 等) 直接生成測資，不靠大量 seed

找漏洞類別及強度

2.
 - Mutation-based: 擅長找輸入的漏洞，buffer overflow，parsing 錯誤等等。不過在複雜巢狀結構不太有效
 - Generation-based: 可以找到比較深層的漏洞，邏輯謬誤，protocol/state 層級的問題

效率

3.
 - Mutation-based: 超快，除了只需要 valid 的 seed，可以每秒生成好幾百萬的測資。不過可能會浪費執行資源 (無效輸入等問題)

- Generation-based: 需要比較高的心力去 setup (需要很了解系統架構以及 input format) ◦

參考資料：

<https://www.we-fuzz.io/blog/generative-vs-mutation-based-fuzzing#:~:text=Mutation-based fuzzing is fast%2C simple%2C and shallow. Generative,limitations%2C but together they provide comprehensive vulnerability discovery.>

(b)

- 拿 AFL 為例子，主要就這 4 步驟：
 1. 用 AFL 的編譯器編譯目標程式的 binary file 以 "instrument" 它
 2. 建造 seed 測資以開始 fuzz: 先做窮舉，可以重現的小變異，path coverage 沒增加的話就做大量隨機的破壞式變異。其中 AFL 會維護一個 queue，包含有用的測資。如果可以增加 path coverage 的就會被標記程 favored。
 3. 在 "instrumented binary" 上跑 AFL: 當它在 process 測資，它會監測哪段的程式被動到 (也就是會追蹤 code paths)。基於這些 path，會變異 seed 檔案來觸發新的 code path，所以 path coverage 會增加
 4. 分析結果

參考資料：

https://blog.csdn.net/weixin_44174312/article/details/154442622

<https://www.fuzzingbook.org/html/GreyboxFuzzer.html>

<https://medium.com/@ayushpriya10/fuzzing-applications-with-american-fuzzy-lop-afl-54facc65d102>

(c)

skip

(d)

skip

5. 敗北協定太多了！

DNS SECURITY

(a)

DNS amplification attack:

1. 攻擊者利用 botnet 向開放的 DNS resolver 提出遞迴請求(UDP)，目標都設定成受害者的真實 IP 位址
 2. 每個 UDP 封包通常傳遞一個參數 (例如“ANY”) 以接收盡可能最大的回應
 3. 這些 DNS resolver 向 root 一層一層查詢，最後把這些流量都倒入目標 IP 位置，導致阻斷服務
- 防止方法：
 - 減少 public DNS resolver 的數量，且讓這些只回應來自受信任來源的查詢
 - 因為攻擊者 botnet 發送的 UDP 一定要有一個偽造成受害者 ip 的 source IP，所以若在內網發送的封包有看起來像來自外網的 ip，那可能是偽造的封包，可被丟棄。(Source address validation)

參考資料：

<https://www.cloudflare.com/zh-tw/learning/ddos/dns-amplification-ddos-attack/>

(b)

DNS cache poisoning:

1. 攻擊者先讓一個遞迴 resolver 去查一個隨機且不存在 (例如 aaa.target.example.) 的子域，迫使 resolver 向該 zone 的 authoritative server 發 query
2. 當 resolver 發 query 時，攻擊者假扮 authoritative server 在網路上快速(幾毫秒)大量發送偽造的 DNS 回應，每個回應都嘗試猜測正確的 transaction ID 與 source port
3. 若某一個偽造回應恰好被 resolver 接受 (transaction ID/port 符合)，resolver 就會把那組惡意的 NS/A 緩存起來
4. 因為緩存的是整個 zone 的 nameserver 資訊，之後任何對該 zone 的查詢都會被遞迴到攻擊者控制的 nameserver

5. 攻擊者控制的 nameserver 可回任意答案，因此攻擊者可以把 zone 內所有記錄 (A、MX、TXT 等) 替換成惡意值，達到「污染整個 zone」的目的

- 防止辦法：
 - DNSSEC，讓 DNS 紀錄能被數位簽證，而 resolver 用公鑰來確認簽章，攻擊者沒有私鑰就無法建造可被接受的 DNS 回應。
 - 把開放的 Resolvers 關掉，只允許內網而已。也可以隨機化埠，避免用很好猜的 port 53。

參考資料：

<https://www.cloudflare.com/zh-tw/learning/dns/dns-cache-poisoning/>

<https://ieeexplore.ieee.org/document/10671556>

NASA HW5

SMTP SECURITY

(c)

- SPF (Sender Policy Framework) 會在郵件寄件者的 DNS 紀錄中增加 SPF 的紀錄，長得像是 v=spf1... 開頭的純文字，包含被授權許可發信伺服器的 IP，收件方可以檢查這個資訊去看發送著 domain 的 DNS record，檢查有沒有存在 SPF 紀錄，去判斷要不要信任這個郵件。防止 email spoofing 的方式就是如果不被信任，就可以丟到垃圾郵件或是擋掉。

參考資料：

<https://blog.cloudmax.com.tw/what-is-spf-and-dkim/>

(d)

- DKIM (DomainKeys Identified Mail) 能夠讓網域的擁有者自動簽署來自其網域的郵件。寄件者若有 DKIM，寄件者則使用私鑰簽署電子郵件的標頭。DKIM 紀錄會儲存網域的公鑰，然後收件者的伺服器可以檢查這個網域來取得寄件者的公鑰，就可以驗證郵件真的是來自於寄件者，而沒有被 Spoof。

參考資料：

<https://www.cloudflare.com/zh-tw/learning/email-security/dmarc-dkim-spf/>

(e)

- 一勞永逸是不可能的，舉例來說，可以結合 Display name spoofing 和 Look-alike Domain 做出很擬真的詐騙信。攻擊者可以在信件的 From 標頭寫上 Apple ID 支援，然後後面的 <> domain 區塊仿造一個跟真實 Apple Support 很類似的域名。SPF+DKIM+DMARC 其實只對直接 spoof 信任的域名有很好的防禦效果而已。

參考資料：

<https://powerdmrc.com/how-to-prevent-email-display-name-spoofing/>

TLS SECURITY

(f)

- TLS cert content: 從憑證檢視器可以看到有:
 - 版本: 通常 v3
 - 序號
 - 憑證簽章演算法: ESDSA · PKCS...
 - 簽發者
 - CN (Common name)
 - O (Organization)
 - OU (Organization Unit)
 - 發行給
 - CN
 - O
 - OU
 - 有效性
 - 發行日期
 - 到期時間
 - 主體公開金鑰資訊
 - 演算法: RSA 加密，橢圓曲線公開金鑰等
 - 公鑰
 - sha-256 指紋

。等等

- CA (Certificate Authority): 是受信任的第三方實體，負責驗證網站/服務的身分，並用自己的私鑰簽發 X.509 憑證。功能是可以確認網址真的屬於申請者，而不是冒牌貨。若網際網路使用者造訪未被CA頒發的憑證所保護的網頁，他們就會收到瀏覽器安全警告。

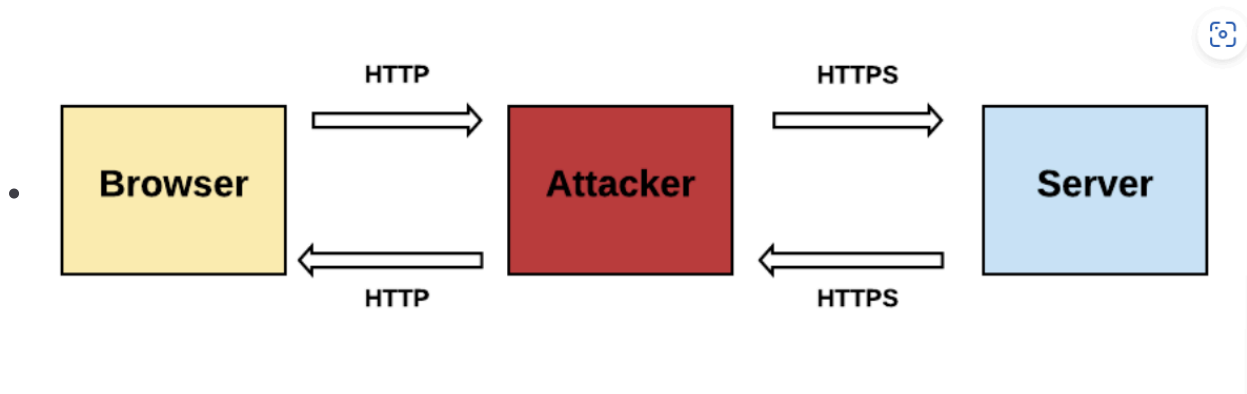
參考資料：

<https://www.cloudflare.com/zh-tw/learning/ssl/transport-layer-security-tls/>

<https://www.digicert.com/tw/faq/compliance/what-is-a-certificate-authority-ca>

(g)

- HTTPS Downgrade Attack: 就如名字所說，攻擊者讓 HTTPS 降階到 HTTP 或是比較弱的 TLS/SSL 版本 (e.g TLS 1.3 -> 1.0)，以竊取或攔截，修改資料。其中有一招是 SSL Stripping，若網站裡有一行像是 `<a href="http://mywebsite.com/awesomepage">Awesome page</a>`，正常來說網頁伺服器會 301 redirect to HTTPS，但如果攻擊者攔截這個 HTTP 請求，中間人攻擊就發生了，他之後就可以看到用戶用明文傳遞的資料，再把這些資料用 HTTPS 送到伺服器。



- How to Prevent: 強制 HTTPS 而且用 HSTS，假如網站是 nginx host 的，那就可以在 sites-available 那邊加入一行 `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload`，這會告訴瀏覽器說之後的連線必須 https。除此之外，也可以在 config 只允許 TLS 1.2 1.3 `ssl_protocols TLSv1.2 TLSv1.3; ssl_prefer_server_ciphers on; ssl_ciphers 'ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:...'`。最後，確保自己的網頁的連結全部用 https://

參考資料：

<https://zh.wikipedia.org/zh-tw/降级攻击>

<https://auth0.com/blog/preventing-https-downgrade-attacks/>

6. 猫物語（赤）

(a)

skip

(b)

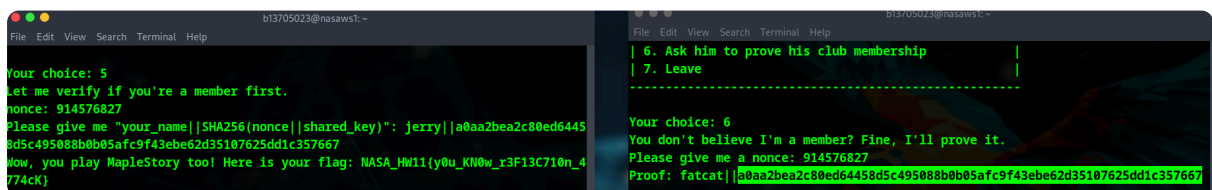
skip

(c)

skip

(d)

- 開兩個終端機連到肥貓
- 因為會員的 key 都是分享的，所以我一個問他 5 一個問她 6
- 問 5 的終端機會拿到一串 nonce，pass 給問 6 的機器
- 接著肥貓會自爆 shared key 出來，就可以拿這個來 pass 進 5 的終端機，人名換一下就好



```
b13705023@nasawsl: ~  
File Edit View Search Terminal Help  
Your choice: 5  
Let me verify if you're a member first.  
nonce: 914576827  
Please give me "your_name|SHA256(nonce|shared_key)": jerry|a0aa2bea2c80ed6445  
8d5c495088b0b05afc9f43ebe62d35107625dd1c357667  
Now, you play MapleStory too! Here is your flag: NASA_HW11{y0u_KN0w_r3F13C710n_4  
774cK}  
  
b13705023@nasawsl: ~  
File Edit View Search Terminal Help  
6. Ask him to prove his club membership  
7. Leave  
-----  
Your choice: 6  
You don't believe I'm a member? Fine, I'll prove it.  
Please give me a nonce: 914576827  
Proof: fatcat|a0aa2bea2c80ed64458d5c495088b0b05afc9f43ebe62d35107625dd1c357667
```

- NASA_HW11{y0u_KN0w_r3F13C710n_4774cK}

參考資料：

https://en.wikipedia.org/wiki/Reflection_attack